LAB EXPERIMENT- 6

Aim: To study and analyze different addressing modes using C programming language programs by compiling with GCC and debugging using GDB in terminal.

Theory :

1. Implied Addressing Mode :

In this mode, the operand is implicitly specified in the instruction itself. No explicit operand is required.

Example: Operations like complement accumulator.

```
  GNU nano 4.8
#include<stdio.h>

int main() {
int x=5 ;
x++ ;
printf("Implied  addressing : x=%d\n",x) ;

return 0 ;
}
```

2. Stack Addressing Mode :

In this mode, operands are stored at the top of the stack. Operations automatically use stack elements.

Example: PUSH and POP operations.

```
  GNU nano 4.8
#include<stdio.h>

int stack[10];
int top = -1;

void push(int x) {
    stack[++top] = x;
}

int pop() {
    return stack[top--];
}

int main() {
    push(10);
    push(20);

    int result = pop() + pop();
    push(result);

    printf("Stack Addressing Mode Result: %d\n", stack[top]);

    return 0;
}
```

3. Immediate Addressing Mode :

In this mode, the actual data (operand) is given directly in the instruction.
Example: MOV R, 10

```
  GNU nano 4.8
#include<stdio.h>

int main() {
int acc=0 ;
acc=acc=10 ;
printf("Immediate: %d\n", acc);

return 0 ;
}
```

4. Direct Addressing Mode :

In this mode, the instruction contains the direct memory address of the operand.
Only one memory access is required.

```
  GNU nano 4.8
#include<stdio.h>

int main() {
int X = 50;
int acc=0 ;
acc = acc + X;    // AC ←AC + [X]
printf("Direct: %d\n", acc);
return 0 ;
}
```

5. Indirect Addressing Mode :

In this mode, the instruction specifies an address that contains another address of the operand.
Two memory accesses are required.

```
  GNU nano 4.8
#include<stdio.h>

int main() {
int y=30 ;
int acc=0 ;
int *ptr=&y ;
acc = acc + *ptr;    // AC ←AC + [[X]]
printf("Indirect: %d\n", acc);

return 0 ;
}
```

6. Register Direct Addressing Mode :

In this mode, the operand is stored in a CPU register, and the instruction refers to that register.
No memory access is required.

```
  GNU nano 4.8
#include<stdio.h>

int main() {
    int acc = 0;
    int R = 40;   // Register

    acc = acc + R;

    printf("Register Direct: %d\n", acc);

    return 0;
}
```

7. Register Indirect Addressing Mode :

In this mode, a register holds the address of the operand instead of the operand itself.
One memory access is required.

```
  GNU nano 4.8
#include<stdio.h>

int main() {
int acc=0 ;
int Z = 40;
int *Rptr = &Z;
acc = acc + *Rptr;
printf("Register Indirect: %d\n", acc);

return 0 ;
}
```

8. Relative Addressing Mode :

In this mode, the effective address is calculated by adding the Program

Counter(PC) value to the address part of the instruction.

```
  GNU nano 4.8
#include<stdio.h>

int main() {
int PC = 1000;
int offset = 20;
int EA = PC + offset;
printf("Relative Address: %d\n", EA);
return 0 ;
}
```

9. Indexed Addressing Mode :

In this mode, the effective address is obtained by adding an index register value to the address part of the instruction.

```
  GNU nano 4.8
#include<stdio.h>
int main() {
int arr[5] = {1,2,3,4,5};
int index = 2;
printf("Indexed: %d\n", arr[index]);

return 0 ;
}
```

10. Base Register Addressing Mode :

In this mode, the effective address is calculated by adding the base register value and the displacement (offset).

```
  GNU nano 4.8
#include<stdio.h>

int main() {
int baseArr[5] = {10,20,30,40,50};
int *base = baseArr;
printf("Base Register: %d\n", *(base + 3));

return 0 ;
}
```

11. Auto-Increment Addressing Mode :

In this mode, after accessing the operand, the register value is automatically incremented.

```
#include<stdio.h>

int main() {
int A[3] = {5,10,15};
int *p = A;
printf("Auto Increment: %d\n", *p++);
printf("Next: %d\n", *p);

return 0 ;
}
```

12. Auto-Decrement Addressing Mode

In this mode, the register value is first decremented, and then the operand is accessed

```
  GNU nano 4.8
#include<stdio.h>

int main() {
int B[3] = {7,14,21};
int *q = &B[2];
printf("Auto Decrement: %d\n", *q--);
printf("Previous: %d\n", *q);

return 0;
}
```

<u>Procedure</u> :

Step 1: Open terminal and create a C program file using nano program.c, then write the code for addressing modes.

Step 2 : Compile the program using GCC with debugging option

Step 3 : Start the debugger using GDB

Step 4 : Set breakpoint and execute the program step-by-step using commands like break main, run, and next.

<u>Observation</u> :

```
nitdelhi@pc:~/Desktop$ nano am.c
nitdelhi@pc:~/Desktop$ gcc -g am.c -o am
nitdelhi@pc:~/Desktop$ gdb am
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from am...
(gdb) run
Starting program: /home/nitdelhi/Desktop/am
Implied  addressing : x=6
[Inferior 1 (process 9208) exited normally]
(gdb) next
The program is not being run.
(gdb) print x
No symbol "x" in current context.
(gdb) print a
No symbol "a" in current context.
(gdb) break 4
Breakpoint 1 at 0x555555555155: file am.c, line 4.
(gdb) run
Starting program: /home/nitdelhi/Desktop/am

Breakpoint 1, main () at am.c:4
4       int x=5 ;
(gdb) next
5       x++ ;
(gdb) continue
Continuing.
Implied  addressing : x=6
[Inferior 1 (process 9255) exited normally]
(gdb)
```

```
nitdelhi@pc:~/Desktop$ gcc -g am_2.c -o am_2
nitdelhi@pc:~/Desktop$ nano am_2.c
nitdelhi@pc:~/Desktop$ gcc -g am_2.c -o am_2
nitdelhi@pc:~/Desktop$ gdb am_2
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from am_2...
(gdb) break 19
Breakpoint 1 at 0x11f9: file am_2.c, line 19.
(gdb) run
Starting program: /home/nitdelhi/Desktop/am_2

Breakpoint 1, main () at am_2.c:19
19      printf("Stack (ADD): %d\n", stack[top]);
(gdb) next
Stack (ADD): 30
20      return 0 ;
(gdb) q
A debugging session is active.

        Inferior 1 [process 9680] will be killed.

Quit anyway? (y or n) y
```

```
nitdelhi@pc:~/Desktop$ nano am_3.c
nitdelhi@pc:~/Desktop$ gcc -g am_3.c -o am_3
nitdelhi@pc:~/Desktop$ gdb am_3
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from am_3...
(gdb) break 4
Breakpoint 1 at 0x1159: file am_3.c, line 4.
(gdb) next
The program is not being run.
(gdb) run
Starting program: /home/nitdelhi/Desktop/am_3

Breakpoint 1, main () at am_3.c:4
4       int acc=0 ;
(gdb) next
5       acc=acc=10 ;
(gdb) continue
Continuing.
Immediate: 10
[Inferior 1 (process 9844) exited normally]
(gdb) q
```

```
nitdelhi@pc:~/Desktop$ nano am_4.c
nitdelhi@pc:~/Desktop$ gcc -g am_4.c -o am_4
nitdelhi@pc:~/Desktop$ gdb am_4
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from am_4...
(gdb) break 4
Breakpoint 1 at 0x1155: file am_4.c, line 4.
(gdb) run
Starting program: /home/nitdelhi/Desktop/am_4

Breakpoint 1, main () at am_4.c:4
4       int X = 50;
(gdb) next
5       int acc=0 ;
(gdb) continue
Continuing.
Direct: 50
[Inferior 1 (process 10111) exited normally]
(gdb) q
```

```
nitdelhi@pc:~/Desktop$ nano am_5.c
nitdelhi@pc:~/Desktop$ gcc -g am_5.c -o am_5
nitdelhi@pc:~/Desktop$ gdb am_5
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from am_5...
(gdb) break 5
Breakpoint 1 at 0x118b: file am_5.c, line 5.
(gdb) run
Starting program: /home/nitdelhi/Desktop/am_5

Breakpoint 1, main () at am_5.c:5
5       int acc=0 ;
(gdb) next
6       int *ptr=&y ;
(gdb) continue
Continuing.
Indirect: 30
[Inferior 1 (process 10359) exited normally]
(gdb) q
```

```
nitdelhi@pc:~/Desktop$ nano am_7.c
nitdelhi@pc:~/Desktop$ gcc -g am_7.c -o am_7
nitdelhi@pc:~/Desktop$ gdb am_7
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from am_7...
(gdb) break 5
Breakpoint 1 at 0x118b: file am_7.c, line 5.
(gdb) run
Starting program: /home/nitdelhi/Desktop/am_7

Breakpoint 1, main () at am_7.c:5
5       int Z = 40;
(gdb) next
6       int *Rptr = &Z;
(gdb) continue
Continuing.
Register Indirect: 40
[Inferior 1 (process 10609) exited normally]
(gdb) q
```

```
nitdelhi@pc:~/Desktop$ nano am_8.c
nitdelhi@pc:~/Desktop$ gcc -g am_8.c -o am_8
nitdelhi@pc:~/Desktop$ gdb am_8
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from am_8...
(gdb) break 5
Breakpoint 1 at 0x115c: file am_8.c, line 5.
(gdb) run
Starting program: /home/nitdelhi/Desktop/am_8

Breakpoint 1, main () at am_8.c:5
5       int offset = 20;
(gdb) next
6       int EA = PC + offset;
(gdb) continue
Continuing.
Relative Address: 1020
[Inferior 1 (process 10979) exited normally]
(gdb) q
```

```
nitdelhi@pc:~/Desktop$ gcc -g am_9.c -o am_9
nitdelhi@pc:~/Desktop$ gdb am_9
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from am_9...
(gdb) break 4
Breakpoint 1 at 0x11a7: file am_9.c, line 4.
(gdb) run
Starting program: /home/nitdelhi/Desktop/am_9

Breakpoint 1, main () at am_9.c:4
4       int index = 2;
(gdb) continue
Continuing.
Indexed: 3
[Inferior 1 (process 11178) exited normally]
(gdb) q
nitdelhi@pc:~/Desktop$
```

```
nitdelhi@pc:~/Desktop$ gcc -g am_10.c -o am_10
nitdelhi@pc:~/Desktop$ gdb am_10
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from am_10...
(gdb) break 6
Breakpoint 1 at 0x11af: file am_10.c, line 6.
(gdb) run
Starting program: /home/nitdelhi/Desktop/am_10

Breakpoint 1, main () at am_10.c:6
6       printf("Base Register: %d\n", *(base + 3));
(gdb) continue
Continuing.
Base Register: 40
[Inferior 1 (process 11385) exited normally]
(gdb) q
```

```
nitdelhi@pc:~/Desktop$ nano am_11.c
nitdelhi@pc:~/Desktop$ gcc -g am_11.c -o am_11
nitdelhi@pc:~/Desktop$ gdb am_11
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from am_11...
(gdb) break 4
Breakpoint 1 at 0x1184: file am_11.c, line 4.
(gdb) run
Starting program: /home/nitdelhi/Desktop/am_11

Breakpoint 1, main () at am_11.c:4
4       int A[3] = {5,10,15};
(gdb) next
5       int *p = A;
(gdb) continue
Continuing.
Auto Increment: 5
Next: 10
[Inferior 1 (process 11567) exited normally]
(gdb) q
```

```
nitdelhi@pc:~/Desktop$ nano am_12.c
nitdelhi@pc:~/Desktop$ gcc -g am_12.c -o am_12
nitdelhi@pc:~/Desktop$ gdb am_12
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from am_12...
(gdb) break 5
Breakpoint 1 at 0x1199: file am_12.c, line 5.
(gdb) run
Starting program: /home/nitdelhi/Desktop/am_12

Breakpoint 1, main () at am_12.c:5
5       int *q = &B[2];
(gdb) continue
Continuing.
Auto Decrement: 21
Previous: 14
[Inferior 1 (process 11710) exited normally]
(gdb) q
```

The program was successfully compiled using GCC with the -g option and executed using GDB. Breakpoints were set at appropriate lines, and the program was executed step-by-step using GDB commands such as run, next, and continue.

During execution, the values of variables and memory locations were observed. Immediate and direct addressing modes showed direct access to values. Indirect and register indirect addressing modes were verified using pointer dereferencing. Indexed, base register, and relative addressing modes were observed using array indexing and pointer arithmetic (base + offset).

Auto-increment and auto-decrement addressing modes were verified by observing changes in pointer values after execution. Stack addressing mode was demonstrated using push and pop operations, where elements were accessed from the top of the stack.

Conclusion :

The different addressing modes were successfully implemented using C programs and analyzed using GDB. The execution of programs confirmed that data can be accessed in various ways such as direct, indirect, register-based, and through pointers and arrays.

The outputs obtained matched the expected results, and the working of each addressing mode was verified through step-by-step debugging. Hence, the experiment was completed successfully.